Hochschule Aalen

Projektbericht
Studiengang: Informatik

XAI-DiabetesCheck

von

Alexander Schweiss

92066

Betreuender Professor: Dr. Mark Herrmann

Einreichungsdatum: 28. Februar 2026

Eidesstattliche Erklärung

Hiermit erkläre ich, **Alexander Schweiss**, dass ich die vorliegenden Angaben in dieser Arbeit wahrheitsgetreu und selbständig verfasst habe.

Weiterhin versichere ich, keine anderen als die angegebenen Quellen und Hilfsmittel benutzt zu haben, dass alle Ausführungen, die anderen Schriften wörtlich oder sinngemäß entnommen wurden, kenntlich gemacht sind und dass die Arbeit in gleicher oder ähnlicher Fassung noch nicht Bestandteil einer Studien- oder Prüfungsleistung war.

28.02.26	A. Schw
Ort, Datum	Unterschrift (Student)

EIGENSTÄNDIGKEITSERKLÄRUNG

Hochschule Aalen

Erklärung über die selbstständige Erbringung einer Prüfungsleistung

Name, Vorname: Schweiss, Alexander	**Semester:** 6
Matrikelnummer: 92066	
Art der Prüfungsleistung: Projektarbeit	**Prüfungsnummer** (lt. Curriculum): 31-57613
Name der Leistung/Veranstaltung: SE - Projekt	

Hiermit versichere ich, dass ich die vorstehende Prüfungsleistung **selbstständig** erbracht und keine anderen als die angegebenen Hilfsmittel benutzt habe.Die Stellen der oben genannten Arbeit, die anderen Werken dem Wortlaut oder dem Sinn nach entnommen wurden, habe ich in jedem einzelnen Fall durch die Angabe der Quelle bzw. der Herkunft als Entlehnung kenntlich gemacht. Dies gilt auch für Zeichnungen, Skizzen, bildliche Darstellungen sowie für Quellen aus dem Internet und anderen elektronischen Text- und Datensammlungen und dergleichen.

Gemäß der mit der prüfenden Person getroffenen Verabredung/Entscheidung bezüglich einer Verwendung generativer Künstlicher Intelligenz („KI“) Systeme gebe ich ergänzend die nachfolgend angekreuzte Versicherung ab.

Zutreffendes bitte ankreuzen:

☐ **1. Verbot generativer KI-Systeme**

Ich versichere, dass ich zur Erstellung der vorliegenden Arbeit keine generativen KI-Systeme genutzt habe.

☒ **2. Erlaubnis der Verwendung generativer KI-Systeme mit Dokumentationspflicht**

Mir ist bekannt, dass mit der Nutzung generativer KI-Systeme keine Garantie für die Qualität von Inhalten und Texten oder anderen Darstellungsformen verbunden ist. Ich verantworte die Übernahme jeglichen Outputs der von mir verwendeten generativen KI-Systeme vollumfänglich selbst. Ich habe alle von der KI generierten Outputs als solche nach den geltenden Zitierregeln ausgewiesen.

In der Anlage „Dokumentation der Verwendung generativer KI-Systeme“ habe ich dargelegt, welche generativen KI-Systeme ich genutzt habe, für welchen Zweck ich diese verwendet habe und auf welche Weise die Nutzung stattfand (z. B. Prompts, Datum der Nutzung, KI-System).

☐ **3. Gebot der Nutzung generativer KI-Systeme mit Dokumentationspflicht**

Ich versichere, dass ich die von der oder dem Erstprüfenden explizit vorgegebenen Tools generativer KI-Systeme genutzt und alle von der KI generierten Outputs als solche nach den geltenden Zitierregeln ausgewiesen habe.

In der Anlage „Dokumentation der Verwendung generativer KI-Systeme“ habe ich dargelegt, für welchen Zweck ich welche Systeme verwendet habe und auf welche Weise die Nutzung stattfand.

Ich bin mir bewusst, dass mit der Nutzung generativer KI-Systeme keine Garantie für die Qualität von Inhalten und Texten oder anderen Darstellungsformen verbunden ist.

Ich verantworte die Übernahme jeglichen Outputs der von mir verwendeten generativen KI-Systeme vollumfänglich selbst.

Die eingereichte Arbeit ist nicht anderweitig als Prüfungsleistung verwendet worden oder in deutscher oder in einer anderen Sprache als Veröffentlichung erschienen. Ich versichere weiterhin, dass ggf. direkt an die Prüfenden zur Begutachtung übermittelte gedruckte Exemplare mit der im Prüfungsamt eingereichten elektronischen Fassung der Arbeit übereinstimmen.

Mir ist bewusst, dass wahrheitswidrige Angaben eine Täuschung darstellen.

Datum, Unterschrift

27.02.26 A. Schw

Studierende Person

Kurzfassung

Im Rahmen meiner Projektarbeit im Studiengang Informatik mit dem Schwerpunkt Software Engineering beschäftigte ich mich mit dem Thema Explainable Artificial Intelligence (AI), oder auch Explainable AI (XAI) genannt.

Das Ziel war, eine Web-Anwendung zu entwickeln, die personenbezogene Daten eines Anwenders nutzt, um dessen Risiko, an Typ-2-Diabetes zu erkranken, vorherzusagen. Das Risiko soll dabei in einem exakten Wert in Prozent zurückgegeben werden und so den Anwender auf sein persönliches Risiko aufmerksam machen. Ein weiteres Ziel war somit, ein Machine-Learning (ML)-Modell mit entsprechenden Daten zu trainieren.

Ein wichtiger Aspekt der Anwendung soll dabei der Explainable-Ansatz sein. Neben dem Risiko wird dem Nutzer erklärt, wie das Modell auf den genauen Wert kommt und was der Anwender für Möglichkeiten hat, um sein persönliches Risiko zu ändern. Somit sollen die Entscheidungsprozesse des Künstliche-Intelligenz (KI)-Modells für Menschen verständlich und nachvollziehbar gemacht werden.

Neben dem Risiko werden dem Nutzer visuell seine Risiko- und Schutzfaktoren dargestellt. In einer Simulation, der What-If-Analyse, kann der Nutzer dann simulieren, wie sich Änderungen seiner Lebensweise direkt auf sein Risiko auswirken würden.

Die Anwendung dient dabei nur zu Forschungs- und Informationszwecken und ist kein Ersatz für eine medizinische Beratung.

Inhaltsverzeichnis

Eidesstattliche Erklärung i

Kurzfassung iii

Abbildungsverzeichnis vi

Tabellenverzeichnis vii

Abkürzungsverzeichnis viii

1. Projektaufbau 1
- 1.1. Datenquelle . . . 1
- 1.2. Features . . . 2
- 1.3. Systemarchitektur . . . 3

2. ML-Modell 4
- 2.1. Explorative Datenanalyse . . . 4
  - 2.1.1. Auswahl der wichtigsten Features . . . 4
  - 2.1.2. Pandas . . . 4
  - 2.1.3. Numerical Python (NumPy) . . . 5
  - 2.1.4. Matplotlib & Seaborn . . . 5
  - 2.1.5. Gruppen & Abhängigkeiten . . . 6
- 2.2. Model Training . . . 7
  - 2.2.1. Scikit-Learn . . . 7
  - 2.2.2. Joblib . . . 8
  - 2.2.3. Train-Test-Split . . . 8
  - 2.2.4. Random Forest . . . 8
  - 2.2.5. Recall . . . 9
- 2.3. SHapley Additive exPlanations (SHAP)-Analyse . . . 11
- 2.4. Exportierung des Modells . . . 13

3. Backend 14
- 3.1. Aufbau . . . 14
- 3.2. Flask . . . 14
- 3.3. Create App . . . 14
- 3.4. Diabetes Modell . . . 15

3.5. Application Programming Interface (API) 16
3.5.1. Routes 16
3.5.2. Utils 16
3.6. Tests 16

4. Frontend **18**
4.1. Basis & Bibliotheken 18
4.2. Architektur & Aufbau 19
4.3. Datenhaltung & API-Kommunikation 19
4.4. Benutzereingaben & Formular-Design 19
4.5. Visualisierung & interaktive Analyse 20

5. Explainability-Visualisierung (XAI) **21**
5.1. Vorhersage & Risikoklassifizierung 21
5.2. SHAP-Werte 22
5.3. What-If-Analyse 23

6. Evaluation **25**
6.1. Stärken & Verlässlichkeit 25
6.2. Schwächen 26

7. Projekterweiterungen **27**

A. Dokumentation der Verwendung generativer KI-Systeme **28**

Literatur **29**

Abbildungsverzeichnis

1. Korrelationsmatrix 6
2. Verteilung der Zielvariable 7
3. Konfusionsmatrix 10
4. Top 10 Features 11
5. Risiko-Faktoren Analyse 12

1. Vorhersage des Risikos 22
2. SHAP-Darstellung 23
3. What-If-Analyse 24

Tabellenverzeichnis

1.1. Übersicht der Datensatz-Features 2

2.1. Übersicht der exportierten Dateien 13

Abkürzungsverzeichnis

AI Artificial Intelligence iii
XAI Explainable AI iii
KI Künstliche-Intelligenz iii
ML Machine-Learning iii
BRFSS Risk Factor Surveillance System 1
CDC Disease Control and Prevention 1
BMI Body-Mass-Index 4
SHAP SHapley Additive exPlanations iv
EDA Explorative Datenanalyse 4
NumPy Numerical Python iv
API Application Programming Interface v
ROC-AUC Receiver Operating Characteristic Area Under the Curve 9
HTTP Hypertext Transfer Protocol 16
REST Representational State Transfer 13
CORS Cross Origin Resource Sharing 14
URL Uniform Resource Locator 16
UI User Interface 18
DOM Document Object Model 19
CSS Cascading Style Sheets 18

1. Projektaufbau

1.1. Datenquelle

Die Grundlage für das Training des ML-Modells bildet der öffentlich zugängliche Datensatz „Diabetes Health Indicators Dataset“ [1]. Dieser wurde über die Data-Science-Plattform Kaggle bezogen und basiert ursprünglich auf den Risk Factor Surveillance System (BRFSS) der US-amerikanischen Gesundheitsbehörde Disease Control and Prevention (CDC) aus dem Jahr 2015.

Für dieses Projekt wird eine Variante des Datensatzes verwendet, die 256.680 Datensätze mit jeweils 22 Merkmalen (Features) umfasst. Dabei ist das erste Feature binär und gibt an, ob eine Person bereits Typ-2-Diabetes hat, oder nicht. Die restlichen 21 Features dienen als Eingabedaten für das Modell.

Der Datensatz eignet sich gut für das Thema XAI, da die Features für Laien interpretierbar sind. Im Gegensatz zu klinischen Laborwerten, wie z.B. dem Insulinspiegel, die in medizinischen Datensätzen zu finden sind, nutzt dieser Datensatz verhaltensbasierte Indikatoren. Dies ermöglicht eine effektive Anwendung von XAI-Methoden, da der Nutzer so direkte Zusammenhänge zwischen seinem Lebensstil und dem Krankheitsrisiko erkennen kann.

1.2. Features

Die folgende Tabelle listet alle Features des Datensatzes auf.

Tabelle 1.1.: Übersicht der Datensatz-Features

Originales Feature	**Deutscher Name**	**Zulässige Werte**
Diabetes_binary	Diabetes-Status	0 = kein Diabetes, 1 = Prädiabetes/Diabetes
HighBP	Bluthochdruck	0 = Nein, 1 = Ja
HighChol	Hoher Cholesterinspiegel	0 = Nein, 1 = Ja
BMI	Body-Mass-Index	Numerisch (z. B. 12–98)
Smoker	Raucher	0 = Nein, 1 = Ja ($\geq$ 100 Zigaretten)
Stroke	Schlaganfall	0 = Nein, 1 = Ja
HeartDiseaseorAttack	Herzerkrankung	0 = Nein, 1 = Ja
PhysActivity	Körp. Aktivität	0 = Nein, 1 = Ja (letzte 30 Tage)
Fruits	Obstkonsum	0 = Nein, 1 = Ja (täglich)
Veggies	Gemüsekonsum	0 = Nein, 1 = Ja (täglich)
HvyAlcoholConsump	Starker Alkoholkonsum	0 = Nein, 1 = Ja
GenHlth	Allg. Gesundheit	1 = Ausgezeichnet bis 5 = Schlecht
MentHlth	Mentale Gesundheit	0–30 (Tage mit Beschwerden)
PhysHlth	Physische Gesundheit	0–30 (Tage mit Beschwerden)
Age	Alter	1 = 18–24 bis 13 = 80+ Jahre
Education	Bildung	1 = kein Abschluss bis 6 = College
Income	Einkommen	1 = 10k$ bis 8 = >75k$
CholCheck	Cholesterin-Check	0 = Nein, 1 = Ja (letzte 5 Jahre)
AnyHealthcare	Krankenversicherung	0 = Nein, 1 = Ja
NoDocbcCost	Kostenhindernis Arzt	0 = Nein, 1 = Ja
DiffWalk	Gehbehinderung	0 = Nein, 1 = Ja
Sex	Geschlecht	0 = Weiblich, 1 = Männlich

1.3. Systemarchitektur

In diesem Abschnitt wird kurz auf die grundlegende Systemarchitektur eingegangen. Näheres zur konkreten Implementierung der einzelnen Komponenten ist in den jeweiligen Folgekapiteln beschrieben.

Die Architektur ist modular in ein Frontend und ein Backend aufgeteilt.

Für das Frontend, also die Benutzeroberfläche, wurde die JavaScript-Bibliothek React gewählt. Der Vorteil von React liegt in der komponenten-basierten Architekur. Außerdem lassen sich Veränderungen während der Entwicklung sofort auf eine laufende Anwendung übertragen, ohne die Anwendungen neustarten zu müssen.[8]

Für das Backend wurde Flask verwendet. Da die verwendeten Bibliotheken für ML und XAI auf Python basieren, wurde ein Python-gestütztes Backend verwendet. Flask dient als Webframework für die grundlegende Serverstruktur.[13]

Für das ML-Modell wurde Scikit-learn verwendet. Diese Standardbibliothek dient als Basis für klassisches ML in Python. [17].

Um den Black-Box-Charakter der ML-Modelle aufzulösen werden SHAP-Werte verwendet. SHAP basiert auf der kooperativen Spieltheorie und gilt als Standard in der Erklärbarkeit von KI. Die Wahl fiel auf SHAP, da es die Features global und lokal nach Wichtigkeit sortiert. Somit findet für jeden Nutzer eine individuelle Aufschlüsselung statt. [20]

2. ML-Modell

2.1. Explorative Datenanalyse

Die Explorative Datenanalyse (EDA) ist der erste Schritt in der Entwicklung des ML-Modells. Sie dient dazu, die zugrunde liegende Struktur der Daten zu verstehen, Anomalien zu indentifizieren und die relevantesten Features für das Training zu ermitteln. Für die Durchführung der EDA und die Datenvorbereitung kamen in diesem Projekt die Python-Bibliotheken Pandas, NumPy, Matplotlib und Seaborn zum Einsatz. Näheres dazu in diesem Abschnitt. Implementiert wurde die EDA in der Datei (01_data_exploration.ipynb)

2.1.1. Auswahl der wichtigsten Features

Der ursprüngliche Datensatz umfasste neben der Zielvariablen für Diabetes 21 verschiedene Features. Um die Modellkomplexität zu reduzieren und die Erklärbarkeit im Sinne des XAI-Ansatzes zu fokussieren, wurde eine Feature-Selektion durchgeführt. Zunächst wurde eine Korrelationsmatrix erstellt, um die Daten auf Multikollinearität zu prüfen und lineare Zusammenhänge zwischen den Features und der Zielvariablen zu identifizieren.

Basierend auf dieser Analyse wurden die Dimensionen auf die 16 aussagekräftigsten Features reduziert. Zu den ausgewählten Merkmalen gehören unter anderem der Bluthochdruck, der Body-Mass-Index (BMI), die allgemeine Gesundheit sowie Faktoren wie Rauchen und körperliche Aktivität. Die bereinigte Feature-Liste wurde für den späteren Modelltrainingsprozess in einer strukturierten JSON-Datei (selected_features.json) serialisiert und gespeichert. Die top 16 Features lassen sich der Tabelle 1.1 entnehmen.

2.1.2. Pandas

Für das grundlegende Datenhandling wurde die Bibliothek Pandas verwendet [22]. Sie bietet hochperformante Datenstrukturen wie den DataFrame, der eine effiziente tabellarische Analyse ermöglicht.

Im Rahmen der EDA wurde Pandas zunächst für das Einlesen der CSV-Datei sowie für die Datenbereinigung genutzt. Die Analyse zeigte 24.206 identische Datensätze auf, welche durch die Methode *drop_duplicates()* entfernt wurden, um eine Verzerrung des Modells durch redundante Informationen zu verhindern. Ein weiterer Schritt war die Speicheroptimierung. Durch die Konvertierung der numerischen Datentypen von standardmäßigen 64-Bit-Ganzzahlen auf int8- beziehungsweise int16-Formate (da die meisten Wertebereiche kategorisch oder binär sind), konnte der Speicherbedarf des DataFrames reduziert werden. Zudem wurde über Pandas verifiziert, dass der Datensatz keine fehlenden Werte aufweist.

2.1.3. NumPy

NumPy ist die fundamentale Bibliothek für wissenschaftliches Rechnen in Python [12]. Sie bildet das technologische Fundament, auf dem Pandas und die verwendeten Visualisierungsbibliotheken operieren. Es ermöglicht die vektorisierte Verarbeitung der Arrays und sorgt bei der Berechnung der Statistiken – wie beispielsweise der Mittelwerte für die spätere Gruppenanalyse – für die notwendige Performance.

2.1.4. Matplotlib & Seaborn

Um die Verteilungen und statistischen Zusammenhänge zu visualisieren, wurden Matplotlib [21] und die darauf aufbauende, statistische Visualisierungsbibliothek Seaborn [9] eingesetzt.

Zunächst wurde die Verteilung der Zielvariablen mithilfe eines Countplots visualisiert. Für die Variable BMI wurde ein Boxplot generiert, um die Medianverschiebung und Ausreißer zwischen Diabetikern und Nicht-Diabetikern zu erkennen. Für die verbleibenden Merkmale wurden in einem Raster-Layout Barplots erstellt, die den direkten Anteil der Diabeteserkrankungen pro Kategorie visualisieren.

Alle Visualisierungen können im Verzeichnis *ml/results/* abgerufen werden.

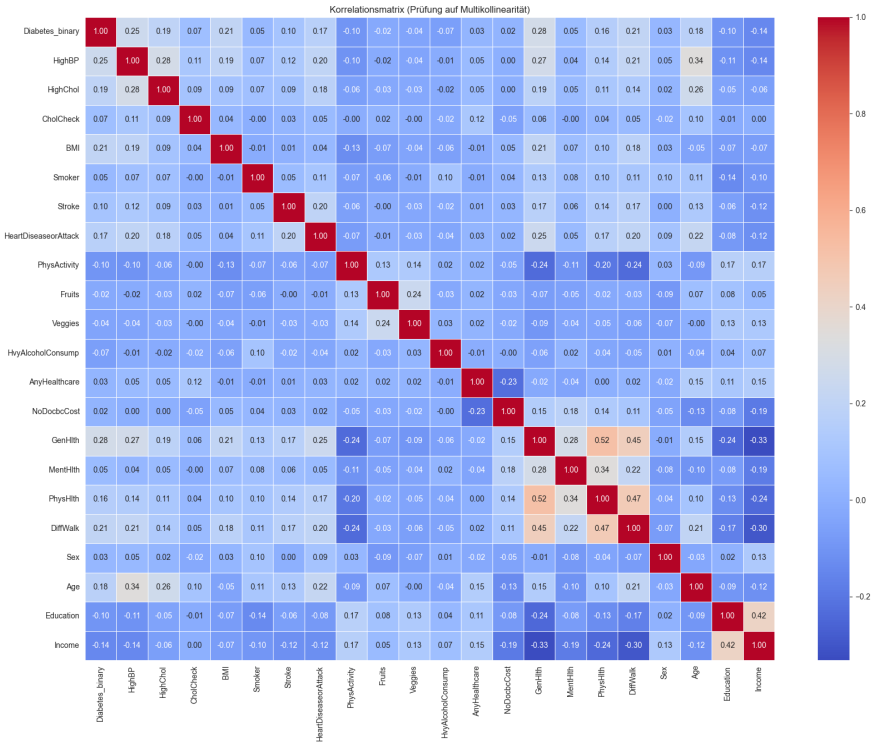

Abbildung 1.: Korrelationsmatrix

2.1.5. Gruppen & Abhängigkeiten

Durch die Kombination aus den Pandas-Aggregationsfunktionen und den Seaborn-Visualisierungen ließen sich Abhängigkeiten in den Gruppen aufdecken.

Die wichtigste Erkenntnis betrifft die Zielvariable. Die Daten weisen ein starkes Klassenungleichgewicht auf. Lediglich 15,3 % der bereinigten Instanzen repräsentieren Personen mit Prädiabetes oder Diabetes, während 84,7 % der Klasse „Kein Diabetes“ angehören.

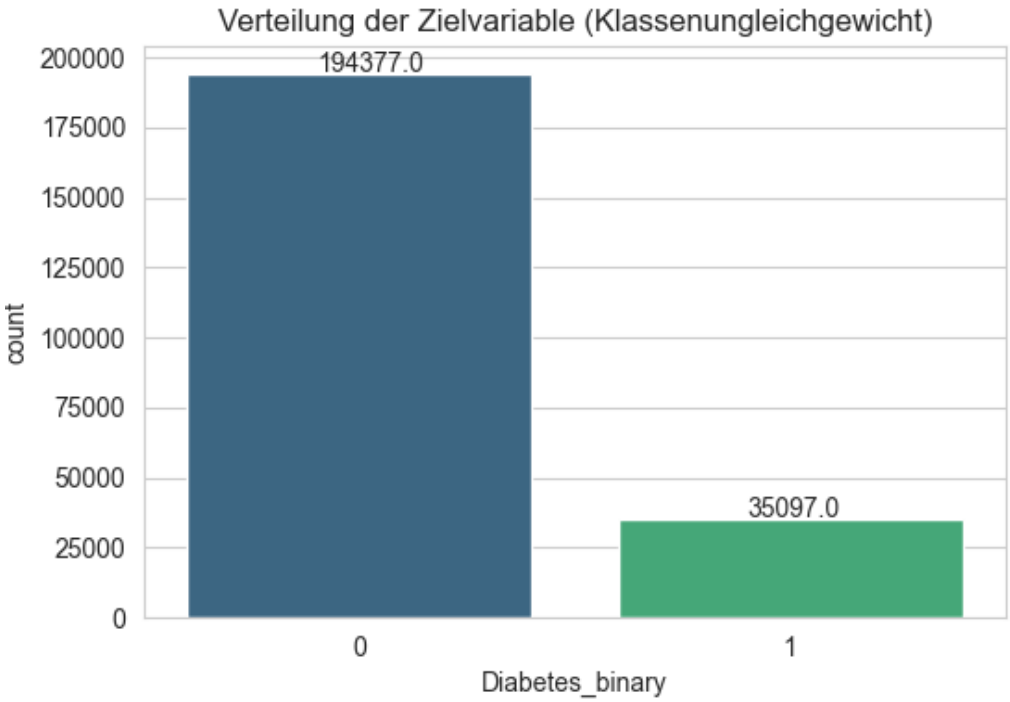

Abbildung 2.: Verteilung der Zielvariable

Die Analyse der gruppierten Features zeigt zudem klare Trends auf. Die Wahrscheinlichkeit einer Diabetes-Klassifikation steigt signifikant bei Personen mit hohem BMI, fortschreitendem Alter sowie Vorerkrankungen wie Bluthochdruck.

2.2. Model Training

In diesem Kapitel wird der Prozess des Modelltrainings beschrieben. Ziel war es, ein Klassifikationsmodell zu entwickeln, das nicht nur eine hohe Gesamtgenauigkeit aufweist, sondern insbesondere im medizinischen Kontext zuverlässige Vorhersagen liefert. Implementiert wurde es in der Datei (02_model_training.ipynb)

2.2.1. Scikit-Learn

Scikit-Learn ist eine weit verbreitete Open-Source-Bibliothek für maschinelles Lernen in Python, die eine einheitliche Schnittstelle für zahlreiche Klassifikations-, Regressions- und Clustering-Verfahren bereitstellt [17]. Scikit-Learn wurde im Projekt für folgende Aufgaben eingesetzt:

- Aufteilung der Daten in Trainings- und Testmenge
- Aufbau einer Pipeline, die eine Skalierung der Merkmale und den eigentlichen Klassifikator kombiniert.
- Durchführung einer Hyperparameter-Optimierung und 3-facher Kreuzvalidierung.
- Berechnung der Evaluationsmetriken sowie die Ausgabe einer Konfusionsmatrix und eines Klassifikationsreports.

2.2.2. Joblib

Für die Speicherung des trainierten Modells wurde die Bibliothek Joblib verwendet. Joblib ist speziell für das effiziente Speichern und Laden von Python-Objekten optimiert, die große numerische Arrays enthalten, wie es bei Scikit-Learn-Modellen der Fall ist. [6] Das gesamte Pipeline-Objekt wird mit Joblib gesichert, sodass es später ohne erneutes Training für Vorhersagen genutzt werden kann. Zusätzlich wurden die Evaluationsmetriken im JSON-Format abgelegt, um einen schnellen Vergleich verschiedener Modellversionen zu ermöglichen.

2.2.3. Train-Test-Split

Um die spätere Generalisierungsfähigkeit des Modells beurteilen zu können, wurde der Datensatz in eine Trainings- und eine Testmenge aufgeteilt. Hierbei kamen 80% der Daten für das Training und 20% für den Test zum Einsatz. Die Aufteilung erfolgte stratifiziert bezüglich der Zielvariable Diabetes, sodass der Anteil der Diabetes-Fälle in beiden Teilmengen gleich bleibt. Das ist wichtig bei unbalancierten Datensätzen, wie sie in in der Medizin häufig vorkommen – in diesem Fall beträgt die Diabetes-Rate im Training nur 13,93%. Eine stratifizierte Aufteilung verhindert, dass die Testmenge zufällig zu wenige oder zu viele positive Beispiele enthält und damit die Ergebnisse der Evaluation verzerrt.

2.2.4. Random Forest

Als Klassifikationsverfahren wurde ein Random Forest gewählt, eine Gruppe aus zahlreichen Entscheidungsbäumen. Die Grundidee besteht darin, viele Bäume auf zufällig gezogenen Stichproben der Daten zu trainieren und bei der Vorhersage den Beschluss der Mehrheit zu verwenden. [18] Random Forests zeichnen sich durch eine hohe Robustheit gegenüber Überanpassung (Overfitting) aus, können nichtlineare

Zusammenhänge modellieren und liefern mit den Wichtigkeiten der Features eine einfache Interpretierbarkeit der Einflussfaktoren.

Im Code wurde eine Pipeline definiert, die zuerst die Merkmale standardisiert und anschließend den *RandomForestClassifier* mit *class_weight='balanced'* anwendet. Die Gewichtung der Klassen gleicht die Ungleichverteilung der Zielvariable aus, indem sie Fehlklassifikationen der Minderheitenklasse stärker bestraft.

2.2.5. Recall

Die gewählte Zielgröße für die Optimierung war der Recall. Der Recall gibt an, wie viele der tatsächlich erkrankten Personen vom Modell korrekt als krank erkannt werden. [19]

In medizinischen Anwendungen ist es oft wichtiger, möglichst wenige erkrankte Personen zu übersehen (False Negatives), selbst wenn dies eine höhere Anzahl an False Positives in Kauf nimmt. Durch die Optimierung auf Sensitivität (Recall) wird das Modell entsprechend angepasst. Das optimierte Modell erreichte auf dem Testdatensatz einen Recall von 0,764, d.h. 76,4% der Diabetes-Fälle wurden korrekt identifiziert. Die Präzision lag mit 0,309 erwartungsgemäß niedriger, da viele Gesunde fälschlich als krank eingestuft wurden. Der resultierende F1-Score betrug 0,440 und die Receiver Operating Characteristic Area Under the Curve (ROC-AUC) 0,821, was auf eine insgesamt gute Trennschärfe hindeutet. Der F1-Score ist dabei ein Wert zwischen 0 und 1 und stellt einen Mittelwert der Präzision und der Sensitivität dar. Je höher der Wert, desto besser. [11] Der ROC-AUC gibt an, wie gut das Modell in der Lage ist, zwei Klassen voneinander zu unterscheiden. Auch dieser Wert liegt zwischen 0 und 1. Je höher der Werte, desto besser das Modell. [23]

Dass auch viele gesunde Personen fälschlicherweise als risikobehaftet eingestuft werden (False Positives), ist im Sinne der angestrebten hohen Sensitivität vertretbar, da die Anwendung nur eine Risikoeinschätzung gibt und keine Diagnose geben soll. Eine Person, die ein Risiko von 80 % hat, kann demnach trotzdem gesund sein und kein Diabetes haben. Die Anwendung zeigt dann aber auf, warum das Risiko so hoch ist.

Die nachfolgenden Abbildungen zeigen die Konfusionsmatrix des finalen Modells sowie die zehn wichtigsten Merkmale, die der Random Forest zur Klassifikation herangezogen hat. Das trainierte Pipeline-Objekt und die Metriken werden für die spätere Verwendung gespeichert.

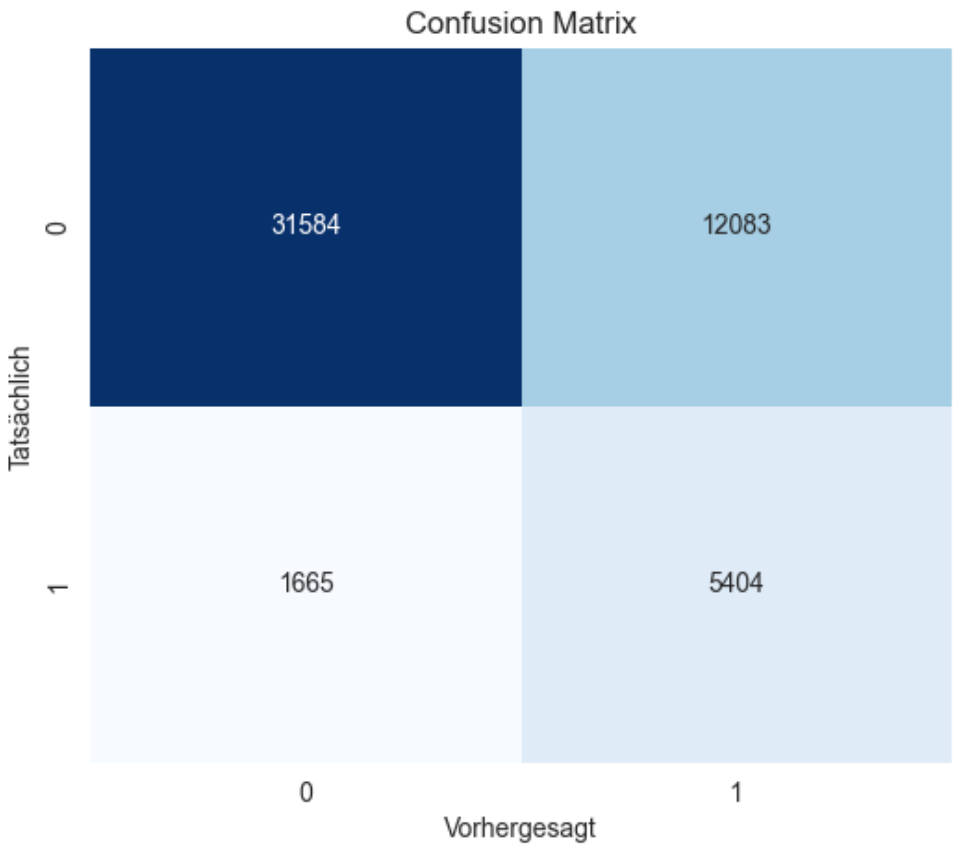

Abbildung 3.: Konfusionsmatrix

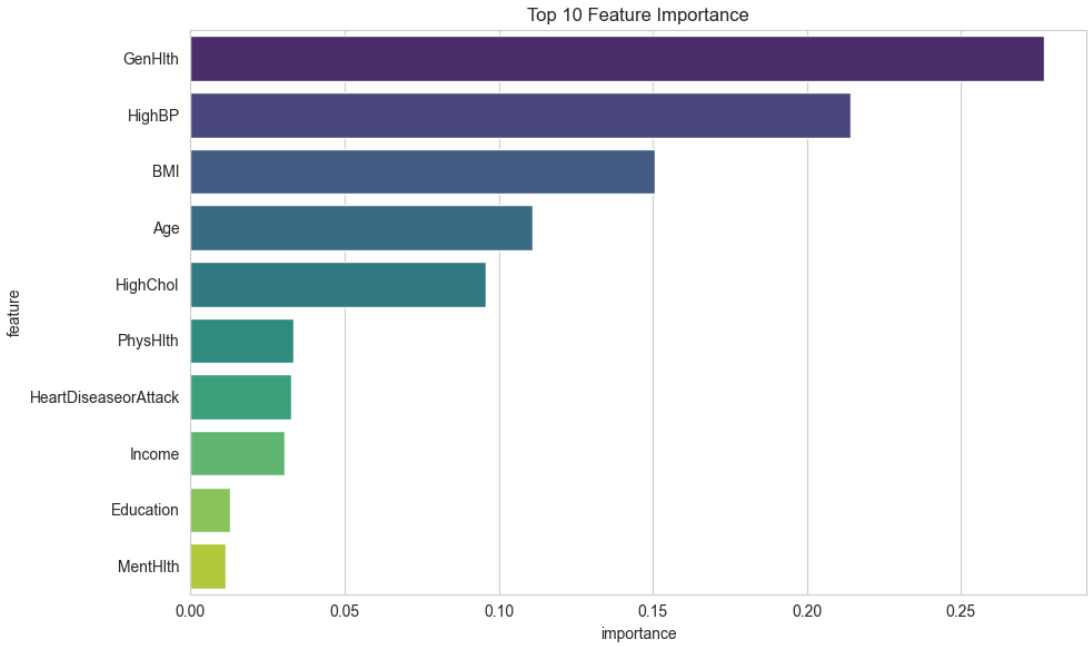

Abbildung 4.: Top 10 Features

2.3. SHAP-Analyse

Um den Black-Box-Charakter des trainierten Modells aufzubrechen und die Vorhersagen für den Anwender transparent zu machen, wird in diesem Projekt das Framework SHAP eingesetzt [20]. SHAP ist ein methodischer Ansatz aus der kooperativen Spieltheorie, der jedem Feature bei einer spezifischen Vorhersage einen exakten Beitragswert (den Shapley-Wert) zuweist. Im Gegensatz zu vielen traditionellen Metriken bietet SHAP den großen Vorteil, dass es sowohl eine globale Erklärbarkeit (wie wichtig ist ein Feature im Gesamtdatensatz) als auch eine lokale Erklärbarkeit (wie stark beeinflusst ein Feature die Risikovorhersage eines individuellen Patienten) ermöglicht. Implementiert wurde dieser Teil in der Datei (03_shap_analyse.ipynb)

Für die Umsetzung wurden das zuvor gespeicherte Modell sowie die Testdaten mithilfe der Bibliothek Joblib und Pandas geladen. Da das Modell innerhalb einer Pipeline trainiert wurde, musste für die SHAP-Analyse eine Vorverarbeitung stattfinden. Der Random Forest erwartet standardisierte Eingabedaten, weshalb die rohen Testdaten zunächst mit dem in der Pipeline integrierten Standard Scaler transformiert wurden. Anschließend wurde der Tree Explainer von SHAP initialisiert. Dieser Algorithmus ist speziell für baumbasierte ML-Modelle optimiert und kann Shapley-Werte für Ensemble-Methoden wie den Random Forest effizient und mathematisch berechnen.

Um die Erklärbarkeit – also die Risikovorhersage für einen individuellen Anwender – zu demonstrieren, wurde exemplarisch ein Waterfall-Plot für einen einzelnen Testpatienten erzeugt. Dieser Plot schlüsselt transparent auf, wie das Modell vom durchschnittlichen Expected Value durch Addition und Subtraktion der individuellen Features zur finalen Vorhersage (z. B. 80 % Diabetesrisiko) gelangt.

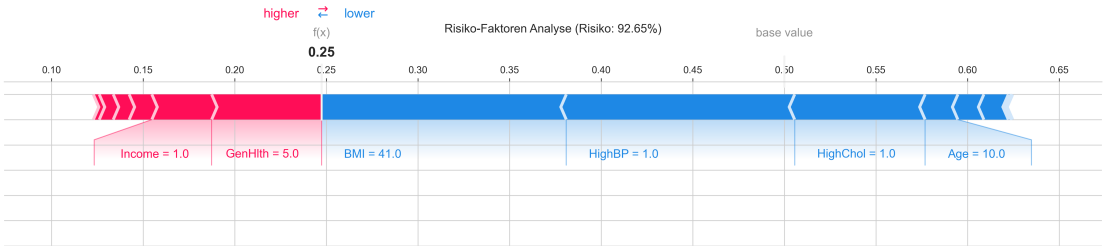

Abbildung 5.: Risiko-Faktoren Analyse

2.4. Exportierung des Modells

Nach der erfolgreichen Evaluation und der Analyse der Features besteht der letzte Schritt der ML-Pipeline in der Exportierung des trainierten Modells. Dieser Vorgang ist erforderlich, um das fertige Modell in das Backend der Web-Anwendung zu überführen und dort für Vorhersagen bereitzustellen [16]. Implementiert wurde dieser Teil in der Datei (04_model_export.ipynb)

Für die Serialisierung der Objekte kommt in diesem Skript erneut die Bibliothek Joblib zum Einsatz. Im Code wurde die Methode *joblib.dump()* aufgerufen, um das vollständig trainierte Modell effizient als binäre .pkl-Dateien abzuspeichern. Der Einsatz von Joblib ist hierbei vorteilhaft, da die erzeugten Entscheidungsbäume des Random Forests große und komplexe NumPy-Arrays enthalten, welche durch Joblib im Vergleich zu nativen Methoden deutlich speicherplatzsparender exportiert werden können.

Zusätzlich zur reinen Vorhersage-Pipeline wurde auch der trainierte SHAP-Explainer exportiert. Dieser Schritt ist wichtig für die Architektur der Web-Anwendung. Da die XAI-Komponente eine Berechnung der Shapley-Werte in Echtzeit erfordert, verhindert das Laden des vorkonfigurierten Explainers im Backend rechenintensive Initialisierungszeiten pro Nutzeranfrage.

Um einen Datenaustausch zwischen dem React-Frontend und dem Modell zu gewährleisten, wurde das integrierte Python-Modul JSON genutzt. [14] Mit *json.dump()* wurde die genaue Liste und Reihenfolge der finalen 16 Features als .json-Datei exportiert. Dies stellt zur Validierung sicher, dass die über die Representational State Transfer (REST)-API eingehenden Anwenderdaten im Backend vor der Übergabe an das Modell in exakt dieselbe Struktur gebracht werden, mit der das Modell ursprünglich trainiert wurde.

Das Ergebnis ist ein portables Verzeichnis mit allen relevanten .pkl- und .json-Artefakten. Diese Dateien werden vom Python-Backend beim Serverstart lediglich noch in den Arbeitsspeicher geladen, um das Diabetesrisiko auf Basis neuer Eingabedaten dynamisch vorherzusagen und zu erklären.

Tabelle 2.1.: Übersicht der exportierten Dateien

Dateiname	**Format**	**Zweck / Inhalt**
diabetes_pipeline	.pkl (Joblib)	Das trainierte Random-Forest-Modell.
shap_explainer	.pkl (Joblib)	Der vorkonfigurierte SHAP-Explainer.
frontend_config	.json	Metadaten für das React-Frontend.
selected_features	.json	Liste der 16 selektierten Features.
metrics	.json	Gespeicherte Performance-Werte.

3. Backend

Das Backend bildet den Hauptteil der Anwendung. Es fungiert als Schnittstelle (API) zwischen dem React-Frontend und dem ML-Modell. Die Hauptaufgaben sind das Entgegennehmen von Nutzerdaten, deren Validierung, die Durchführung der Vorhersage samt SHAP-Analyse und die korrekte Rückgabe der Ergebnisse.

3.1. Aufbau

Die Architektur folgt den Prinzipien einer REST-API. Um den Code wartbar und testbar zu halten, wurde das Backend modular in verschiedene Dateien unterteilt: Die Serverkonfiguration (app.py), die ML-Logik (ml_model.py), das Routing (routes.py) und Hilfsfunktionen (utils.py).

3.2. Flask

Als Fundament wird Flask verwendet. [13] Flask ist ein Webframework für Python, das sich ideal für die Bereitstellung von ML-Modellen eignet, da es keine Ordnerstruktur erzwingt. Zusätzlich kommen Flask-Cross Origin Resource Sharing (CORS) [2] zum Einsatz, damit das Frontend auf einem anderen Port Anfragen an das Backend senden darf, sowie python-dotenv [7], um Umgebungsvariablen wie Host und Port sicher aus einer .env-Datei zu laden.

3.3. Create App

In der Datei app.py wird der Server konfiguriert. Hier wird das sogenannte Application Factory Pattern über die Funktion create_app() angewendet. Das bedeutet, die Flask-App wird nicht global, sondern innerhalb einer Funktion instanziiert. Das hat den Vorteil, dass die Anwendung später leichter getestet werden kann.

In der Datei wird folgendes implementiert:

- Die CORS-Richtlinien werden definiert, sodass nur das eigene Frontend (z. B. auf Port 3000) Zugriffsrechte hat.
- Die maximal erlaubte Größe für Anfragen (Payload) wird limitiert, um den Server vor Überlastung zu schützen.
- Es werden globale Error-Handler eingerichtet, die bei einem falschen Pfad oder einem Serverabsturz keine kryptischen HTML-Seiten, sondern JSON-Fehlermeldungen zurückgeben.
- Ein /health-Endpunkt wird bereitgestellt, der den Status healthy zurückgibt, um die Erreichbarkeit überprüfen zu können.

3.4. Diabetes Modell

Die Datei ml_model.py ist der Hauptteil der API. Hier ist die Klasse *DiabetesModell* definiert, die beim Serverstart genau einmal instanziiert wird.

- Initialisierung: Beim Start lädt die Klasse die Pipeline, den SHAP-Explainer und die Frontend-Konfiguration in den Arbeitsspeicher.
- Validierung (*validate_input*): Bevor Daten an das Modell gehen, werden sie hier geprüft. Fehlen in der Anfrage bestimmte Angaben (z. B. der Cholesterinwert), füllt die Funktion diese mit definierten Standardwerten auf. Zudem prüft sie die Wertebereiche: Ein eingegebener BMI von 200 würde hier abgefangen und eine spezifische Fehlermeldung generieren, bevor das Modell zu falschen Ergebnissen kommt.
- Vorhersage (*predict*): Sind die Daten valide, wird die Wahrscheinlichkeit für Diabetes berechnet. Gleichzeitig berechnet der SHAP-Explainer die individuellen Einflussfaktoren. Die Funktion sortiert diese Faktoren sofort nach ihrer Stärke und trennt sie in risikosteigernde (positive) und risikosenkende (negative) Werte auf.

3.5. API

Die eigentlichen Schnittstellen nach außen sind in zwei Dateien aufgeteilt:

3.5.1. Routes

In routes.py werden mithilfe von Flask-Blueprints die Uniform Resource Locator (URL)-Pfade unter dem Präfix /API definiert.

- Der Endpunkt GET /features liefert dem Frontend die benötigten Strukturdaten, also welche Slider mit welchen minimalen/maximalen Werten angezeigt werden müssen.
- Der Endpunkt POST /predict nimmt die JSON-Eingaben des Nutzers entgegen, ruft die Validierungs- und Vorhersagemethoden aus ml_model.py auf und formatiert die Ergebnisse (Risiko in Prozent, Risikokategorie und die aufgeschlüsselten SHAP-Werte) in eine JSON-Antwort.

3.5.2. Utils

utils.py enthält ausgelagerte Hilfsfunktionen, um die Routen übersichtlich zu halten. Dazu gehört eine Funktion, die die mathematischen SHAP-Ergebnisse für das Frontend verständlich formatiert (z. B. das Runden von Werten und das Labeln von positiv und negativ). Außerdem findet sich hier eine Logging-Funktion, die Anfragen und Ergebnisse in der Konsole protokolliert, dabei aber keine Daten sichtbar macht, indem sie darauf verzichtet, die sensiblen Gesundheitsdaten (Features) der Nutzer zu speichern.

3.6. Tests

Um die Zuverlässigkeit des Backends zu garantieren, wurde das Framework pytest [4] für automatisierte Tests integriert. Das Skript simuliert einen Client, der Hypertext Transfer Protocol (HTTP)-Anfragen an das Backend stellt, ohne den Server real starten zu müssen. Getestet werden verschiedene Szenarien:

- Pfad: Eine Anfrage mit vollständigen und korrekten Daten muss einen Risikowert größer null zurückgeben.
- Edge Cases: Es wird getestet, ob die API bei unrealistischen Eingaben (z. B. BMI von 200) korrekt mit einem Statuscode 400 (Bad Request) und einer

Fehlerliste antwortet.

- Fehlertoleranz: Unvollständige Daten oder komplett ungültige JSON-Strings (z. B. einfacher Text) werden simuliert, um sicherzustellen, dass das Backend die Fallback-Mechanismen nutzt oder den Fehler abfängt, anstatt abzustürzen.

4. Frontend

Das Frontend bildet die visuelle und interaktive Ebene der Anwendung. Es soll die medizinischen und statistischen Daten, wie das Diabetes-Risiko und die SHAP-Werte, in eine zugängliche, leicht verständliche und reaktionsschnelle Benutzeroberfläche übersetzen.

4.1. Basis & Bibliotheken

Als Basis für die Benutzeroberfläche wurde React [8] gewählt. React ist eine deklarative JavaScript-Bibliothek zur Erstellung von Benutzeroberflächen, die auf einem komponenten-basierten Ansatz setzt. Dies ermöglicht es, die User Interface (UI) in isolierte, wiederverwendbare Bausteine zu zerlegen. Außerdem lassen sich Anpassungen im Code während der Laufzeit anwenden, was das Entwickeln effizient gestaltet. Ursprünglich war geplant das Aussehen des Frontends mit Tailwind [5] zu gestalten. Dies wurde nach einiger Zeit verworfen, da Tailwind Probleme verursacht hat. Dabei wurden Cascading Style Sheets (CSS) Klassen nicht gefunden, was das Gestalten letztendlich erschwerte. Alternativ wurde auf normales Inline-Styling zurückgegriffen.

Zur Erweiterung der Funktionalität kommen weitere Bibliotheken zum Einsatz:

- Axios [24]: Ein HTTP-Client, der für die asynchrone Kommunikation mit der Backend-API genutzt wird. Er vereinfacht das Senden von JSON-Daten und das Abfangen von Fehlern.
- Recharts [15]: Eine auf D3.js [10] basierende Bibliothek für React. Sie wird verwendet, um die komplexen SHAP-Ergebnisse als optisch ansprechende und reaktive Balkendiagramme zu visualisieren.
- React-Icons [3]: Eine Bibliothek, die gängige Icon-Sammlungen bündelt, um das UI optisch aufzuwerten und Icons und Frontend-Bausteine (z. B. Ladesymbole, Pfeile) bereitzustellen.

4.2. Architektur & Aufbau

Die Architektur der Frontend-Anwendung folgt dem Container-Presenter-Pattern (oder Smart/Dumb Components).

Der Einstiegspunkt der Anwendung ist die Datei main.jsx, welche die Anwendung in den Document Object Model (DOM) des Browsers einhängt. Die eigentliche Verwaltung übernimmt die App.jsx. Diese Datei fungiert als globaler Verwalter der Zustände. Sie hält die Formulardaten, den Ladestatus und die Ergebnisse der API-Anfrage im Speicher. Beim initialen Laden prüft die App.jsx zudem die Erreichbarkeit des Backends (health check), um dem Nutzer bei Verbindungsproblemen frühzeitig eine Rückmeldung zu geben.

Abhängig vom Zustand (z. B. „Daten werden geladen“ oder „Ergebnis liegt vor“) rendert die App.jsx bedingt die jeweiligen Komponenten. Diese Unterkomponenten erhalten ihre Daten ausschließlich über Properties von der Hauptkomponente und sind dadurch wiederverwendbar und testbar.

4.3. Datenhaltung & API-Kommunikation

Um die Backend-Logik von der UI-Logik zu trennen, sind alle Backend-Aufrufe in einer Servicedatei (api.js) gekapselt. Hier wird eine zentrale Axios-Instanz mit der Basis-URL des Backends konfiguriert.

Die Datei stellt Funktionen zum Abrufen der Feature-Daten und zum Absenden der Vorhersage-Daten (*predictDiabetes*) bereit. Ein wesentlicher Bestandteil hierbei ist das Error-Handling: Antwortet das Backend mit einem Fehler (z. B. Validierungsfehler) oder ist der Server offline, fängt die api.js diese Zustände ab und generiert saubere Fehlermeldungen, die in der Benutzeroberfläche ausgegeben werden können.

4.4. Benutzereingaben & Formular-Design

Die Eingabe der Nutzerdaten erfolgt über die Komponente FeatureForm.jsx. Anstatt jedes Eingabefeld (z. B. Alter, BMI, Bluthochdruck) statisch in den Code zu schreiben, wird das Formular dynamisch generiert.

Unterstützt wird dieser Prozess durch ausgelagerte Konfigurationsdateien:

- constants.js: Beinhaltet die technischen Grenzwerte (Min/Max) und Standardwerte für jedes Feature.

- formatters.js: Beinhaltet die Lokalisierung. Hier werden die technischen englischen Feature-Namen in nutzerfreundliche deutsche Labels (z. B. „Bluthochdruck“) und leicht verständliche Beschreibungen für Tooltips übersetzt.

Die Formular-Komponente iteriert über diese Konfigurationen und rendert entsprechende Eingabefelder. Dies hat den Vorteil, dass bei einer Änderung des ML-Modells (z. B. Hinzufügen eines neuen Features) lediglich die Konstanten angepasst werden müssen, ohne die UI-Logik selbst umzubauen.

4.5. Visualisierung & interaktive Analyse

Sobald die Vorhersage des Backends erfolgreich in der App.jsx ankommt, werden drei Komponenten zur Ergebnisanalyse sichtbar:

1. ResultDisplay.jsx: Diese Komponente bricht das Gesamtergebnis herunter. Eine Hilfsfunktion aus den formatters.js weist dem prozentualen Risiko eine von drei Farben (Grün für niedrig, Gelb für mittel, Rot für hoch) zu. Zusätzlich listet die Komponente die Top-Faktoren auf, die das Risiko am stärksten erhöhen oder senken. Diese Faktoren werden aus den SHAP-Werten gewonnen.

2. ShapVisualization.jsx: Hier wird Recharts verwendet. Alle vom Backend gelieferten SHAP-Werte werden in einem interaktiven Balkendiagramm dargestellt. Positive (risikosteigernde) Werte werden rot, negative (risikosenkende) Werte grün gezeichnet. Tooltips geben beim Hovern mit der Maus genaue Auskunft über den exakten Einflusswert jedes Features.

3. WhatIfAnalysis.jsx: Diese Komponente bietet den größten interaktiven Mehrwert (Was-wäre-wenn-Analyse). Sie übernimmt die initial eingegebenen Daten des Nutzers und erlaubt es diesem, gezielt Parameter zu verändern – zum Beispiel den BMI zu reduzieren oder den Raucherstatus zu ändern. Mit einem Klick auf „Simulation starten“ wird eine erneute API-Anfrage gesendet und das neue niedrigere, oder höhere Risiko wird berechnet. Dies fördert das Verständnis des Nutzers für seine eigenen Gesundheitsdaten und motiviert zu präventiven Maßnahmen.

Näheres zu der Explainability-Visualisierung im nächsten Kapitel.

5. Explainability-Visualisierung (XAI)

Der Einsatz von ML-Modellen im Gesundheitswesen scheitert noch immer am Vertrauen. Wenn ein System lediglich eine abstrakte Wahrscheinlichkeit ausgibt („60 % Risiko“), bleibt für den Nutzer oder behandelnden Arzt unklar, wie diese Entscheidung zustande kam (Black-Box-Problem). Das Ziel dieses Projektes ist es daher, durch gezielte XAI-Methoden Transparenz zu schaffen.

Die Benutzeroberfläche wurde so konzipiert, dass sie lokal interpretiert werden kann – sie erklärt also nicht das Modell im Allgemeinen, sondern liefert Erklärungen für die individuellen Eingabedaten eines Nutzers. Dieser Erklärungsprozess ist in drei visuelle Bereiche unterteilt.

5.1. Vorhersage & Risikoklassifizierung

Der erste Bereich der Visualisierung dient der schnellen, intuitiven Erfassung des Gesamtergebnisses. Die vom Backend berechnete Wahrscheinlichkeit für das Diabetes-Risiko (z. B. 65 %) wird dem Nutzer nicht nur als Prozentzahl präsentiert, sondern in mehreren Varianten.

- Ampelsystem: Mithilfe einer in formatters.js definierten Logik wird der Prozentwert in eine von drei Risikokategorien (Niedrig, Mittel, Hoch) übersetzt und farblich codiert (Grün, Gelb, Rot). Dies reduziert die kognitive Last für den Anwender, da die Schwere des Ergebnisses auf den ersten Blick erfasst werden kann.
- Top-Treiber: Direkt neben dem Gesamtrisiko werden die entscheidendsten Einflussfaktoren in Textform aufgelistet („Risikofaktoren“ und „Schützende Faktoren“). Dies beantwortet die unmittelbarste Frage des Nutzers: „Warum ist mein Risiko so hoch/niedrig?“ auf eine erstmal einfache Weise.

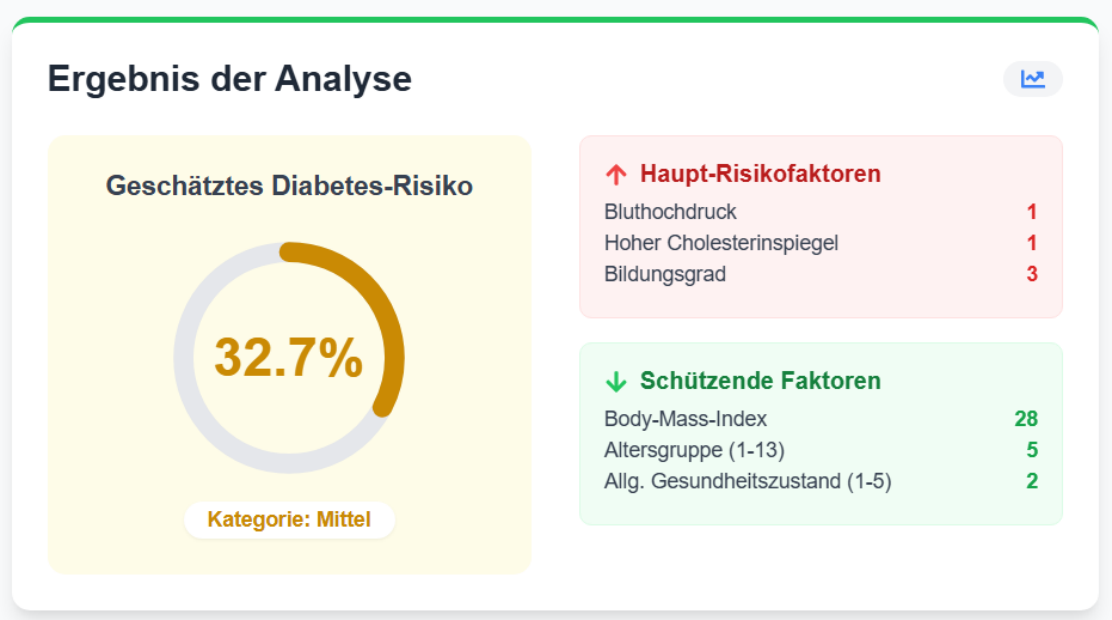

Abbildung 1.: Vorhersage des Risikos

5.2. SHAP-Werte

Während der erste Bereich nur die stärksten Faktoren benennt, erklärt die Komponente ShapVisualization.jsx alle Werte nochmal genauer. Hier wird die vom Backend berechnete SHAP-Analyse visuell dargestellt.

Die Darstellung erfolgt als Balkendiagramm:

- Rote Balken (Positive SHAP-Werte): Diese weisen nach rechts und signalisieren Features, die das Risiko des Nutzers im Vergleich zum Durchschnitt erhöhen (z. B. ein hoher BMI oder Bluthochdruck).
- Grüne Balken (Negative SHAP-Werte): Diese weisen nach links und visualisieren schützende Faktoren, die das individuelle Risiko senken (z. B. viel körperliche Aktivität oder ein junges Alter).

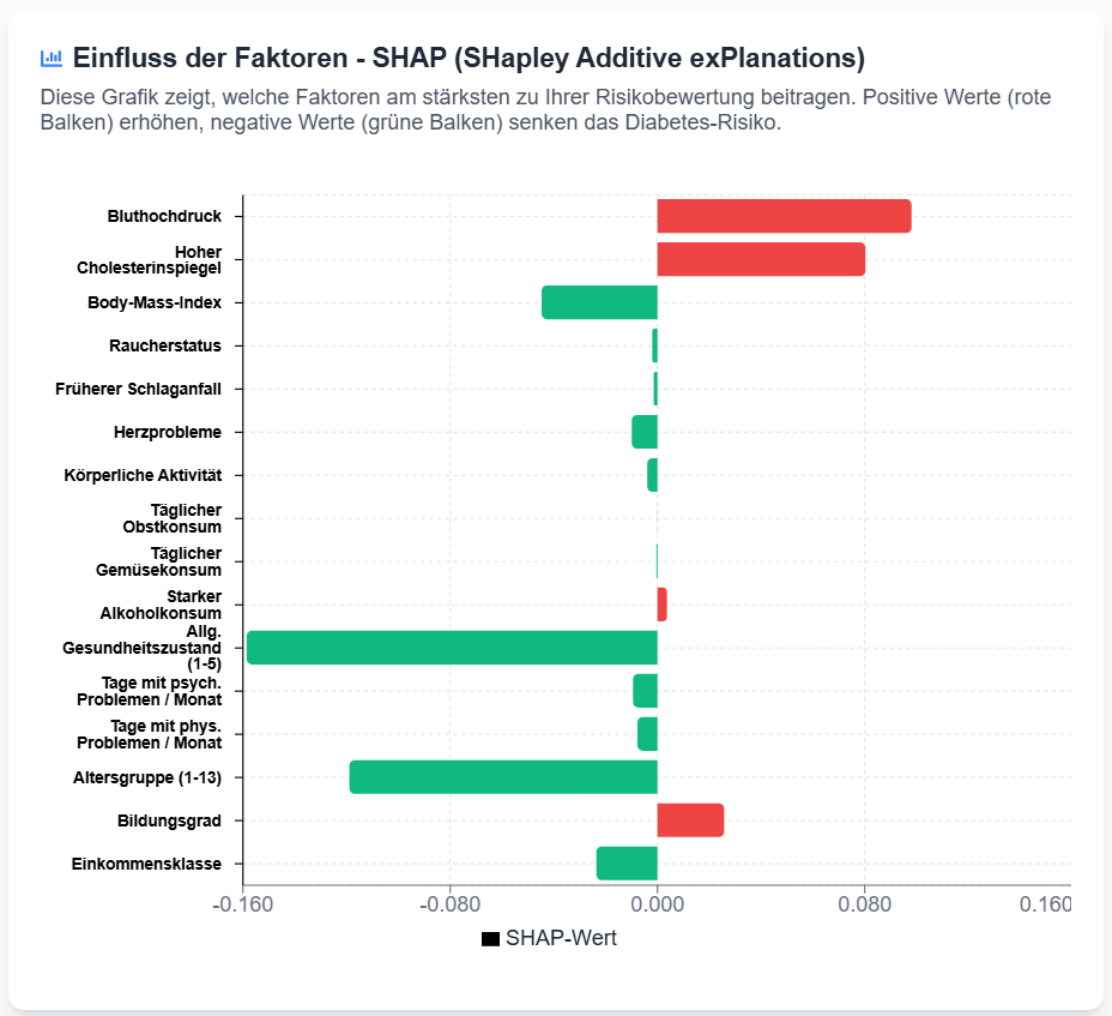

Abbildung 2.: SHAP-Darstellung

Durch die Sortierung der Balken nach ihrer absoluten Auswirkung entsteht eine Rangfolge. Der Nutzer erkennt sofort, welche Lebensgewohnheiten oder gesundheitlichen Parameter den stärksten Einfluss auf sein Risiko haben. Tooltips, die beim Überfahren der Balken erscheinen, liefern zudem den exakten Einflusswert, was die Transparenz weiter erhöht.

5.3. What-If-Analyse

Eine noch bessere Erklärbarkeit wird erreicht, wenn das System nicht nur beschreibend, sondern interaktiv und handlungsorientiert ist. Genau dies setzt die

Komponente WhatIfAnalysis.jsx um.

Eine Erklärung für ein hohes Risiko, aufgrund eines hohen BMIs ist zwar transparent, aber deterministisch. Die What-If-Analyse ermöglicht es dem Nutzer, „Was-wäre-wenn“-Fragen an das Modell zu stellen: Z. B. was passieren würde, wenn der BMI auf einen Wert von 25 gesenkt werden würde.

- Interaktive Simulation: Die Komponente übernimmt die initialen Eingabedaten des Nutzers. Der Anwender kann nun einzelne Parameter über Slider verändern und eine neue Vorhersage beim Backend anfragen.
- Präventiver Mehrwert: Das Frontend stellt das ursprüngliche und das neu simulierte Risiko gegenüber. Dies verdeutlicht die Nicht-Linearität des ML-Modells (z. B. dass eine Gewichtsabnahme, kombiniert mit anderen Faktoren, das Risiko senken kann).

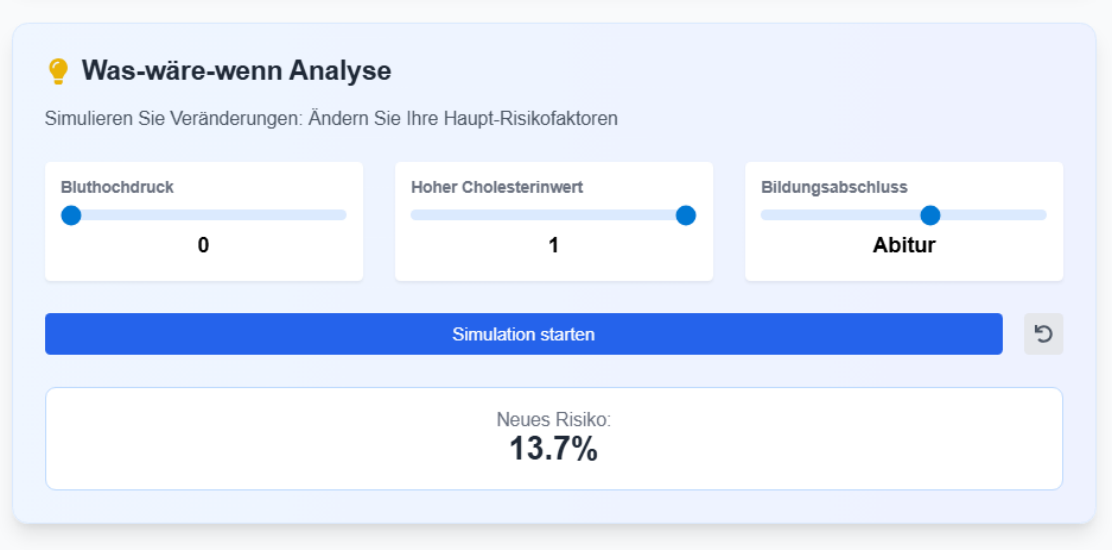

Abbildung 3.: What-If-Analyse

Diese Form der Visualisierung transformiert das ML-Modell von einem Diagnose-Werkzeug zu einem besseren Tool für Prävention. Es fördert das Vertrauen in die KI, da das Modell berechenbar auf veränderte Eingaben reagiert und dem Nutzer Handlungsspielräume aufzeigt.

6. Evaluation

Die Evaluation des entwickelten Systems beinhaltet die quantitative Bewertung des trainierten ML-Modells und die qualitativen Aspekte im Bereich der medizinischen Anwendbarkeit und Erklärbarkeit.

6.1. Stärken & Verlässlichkeit

Die Evaluierung auf dem isolierten Testdatensatz zeigt, dass das Random-Forest-Modell über eine insgesamt gute Trennschärfe verfügt. Die ROC-AUC erreicht einen Wert von 0,821. Der ROC-AUC gibt an, wie gut das Modell in der Lage ist, zwei Klassen voneinander zu unterscheiden. Dieser Wert liegt zwischen 0 und 1. Je höher der Wert, desto besser und zuverlässiger ist das Modell. Ein Wert von über 0,8 belegt, dass das Modell in der Lage ist, valide Muster zu erkennen.

Hervorzuheben ist die Leistungsfähigkeit des Modells im Hinblick auf die Sensitivität. Das Modell erreichte einen Recall von 0,764, das heißt, 76,4 % der tatsächlichen Diabetes-Fälle wurden korrekt erkannt. Im Kontext eines medizinischen Assistenz- und Präventionssystems ist ein hoher Recall eine wichtige Metrik. Die Kosten für ein False Negative – also das Übersehen einer Person, die tatsächlich ein hohes Diabetesrisiko hat – sind ungleich höher als die eines falschen Alarms. Das System agiert somit konservativ und sicherheitsorientiert. Zudem ist eine Diagnose nicht das Ziel der Anwendung, lediglich eine Risikoeinschätzung. Somit sind Fehldiagnossen nicht schlimm, da auch eine Person mit einem Diabetes-Risiko von 80 % noch gesund sein kann und lediglich ein hohes Risiko hat, noch zu erkranken. Die Anwendung dient nur zu Präventionszwecken und ist kein Ersatz für eine medizinische Beratung.

Eine weitere Stärke und ein Faktor für die Verlässlichkeit der Anwendung ist der XAI-Ansatz durch SHAP. Klassische Random-Forest-Modelle gelten als "Black Box". Durch die Integration der What-If-Analyse und der visuellen SHAP-Auswertung wird das Modell transparent. Dem Nutzer wird logisch dargelegt, warum ein Risiko prognostiziert wurde. Dies stärkt das Vertrauen in die KI, da Vorhersagen nicht blind akzeptiert werden müssen, sondern anhand nachvollziehbarer Parameter überprüfbar sind.

6.2. Schwächen

Die Fokussierung auf eine hohe Sensitivität (Recall) bringt Kompromisse mit sich. Die Präzision liegt mit 0,309 niedriger, da viele eigentlich gesunde Anwender durch das Modell fälschlicherweise als risikobehaftet (krank) eingestuft wurden (False Positives), da nur ihr Risiko sehr hoch ist.

Der resultierende F1-Score betrug 0,440. Der F1-Score ist ein Wert zwischen 0 und 1 und stellt das Mittel aus Präzision und Sensitivität dar (je höher der Wert, desto besser) [11]. Der niedrige F1-Score zeigt den starken Trade-off zugunsten der Sensitivität. Somit stellt die geringe Präzision eine praktische Schwäche dar: Eine zu hohe Rate an Fehlalarmen kann bei Endanwendern zu unbegründeten Sorgen führen, sodass Warnungen der App nach mehrfacher Nutzung nicht mehr ernst genommen werden.

Eine weitere Schwäche liegt in den zugrunde liegenden Daten. Der verwendete CDC-Datensatz (BRFSS) basiert auf mündlichen Befragungen. Solche Datensätze unterliegen bekannten Verzerrungen. Nutzer neigen z.B. dazu, ihren Alkoholkonsum zu unterschätzen oder ihre körperliche Aktivität zu überschätzen.

Außerdem stehen dem Modell keine klinischen Parameter zur Verfügung. Während ein Arzt für eine echte Risikovorhersage Blutzuckerwerte heranziehen würde, verlässt sich die Anwendung rein auf verhaltensbasierte und soziale Indikatoren. Dies ist für die interaktive What-If-Analyse des Lebensstils ideal, limitiert jedoch die diagnostische Genauigkeit des Modells und betont die Notwendigkeit des in der Fußzeile integrierten Hinweises, dass die Anwendung keinesfalls eine professionelle medizinische Beratung ersetzen kann.

7. Projekterweiterungen

Der aktuelle Stand des XAI-DiabetesCheck zeigt, dass Machine Learning in Kombination mit XAI (SHAP) einen Beitrag zur medizinischen Prävention leisten kann. Um die Anwendung weiter zu verbessern, bieten sich verschiedene Erweiterungen auf Daten-, Modell- und Anwendungsebene an.

Die größte Limitation des aktuellen Systems ist die Abhängigkeit von subjektiven Umfragedaten (BRFSS-Datensatz). Eine Erweiterung wäre die Integration objektiver, klinischer Parameter. Durch die Einbeziehung von Laborwerten wie verschiedene Blutzuckerwerten ließe sich die Präzision deutlich steigern.

Im Bereich der Softwarearchitektur sind ebenfalls funktionale Erweiterungen denkbar:

- Langzeit-Tracking (Benutzer Accounts): Die Einführung einer Datenbank und von Benutzerkonten würde es ermöglichen, Risikoanalysen über die Zeit zu speichern. Nutzer könnten so in einem Dashboard sehen, wie sich Lebensstilanpassungen über Wochen und Monate auf ihr Diabetesrisiko auswirken.
- Batch-Verarbeitung für Kliniken: Im Flask-Backend kann ein Batch-Endpunkt angelegt werden. Die Ausarbeitung dieser Funktion würde es medizinischem Personal ermöglichen, Datensätze von hunderten Patienten gleichzeitig hochzuladen und auszuwerten, um Risikogruppen für Vorsorgeuntersuchungen zu identifizieren.

A. Dokumentation der Verwendung generativer KI-Systeme

Das KI-Sprachmodell Google Gemini wurde als unterstützendes Werkzeug eingesetzt. Die Nutzung der generativen KI diente dabei nicht der grundlegenden Architekturentwicklung oder der inhaltlichen Texterstellung, sondern fokussierte sich primär auf die technische Problemlösung, das Debugging und die Code-Transformation. Im Detail wurde die KI in folgenden Projektphasen herangezogen:

Machine-Learning-Modell
Bei der Entwicklung der ML-Pipeline (Datenverarbeitung, Modelltraining und SHAP-Integration) wurde Gemini zur gezielten Fehlersuche eingesetzt. Dies umfasste hauptsächlich die Analyse von Fehlermeldungen/Exceptions, wenn nach eigener Suche keine Lösungen gefunden werden konnten.

Backend-Entwicklung
Im Bereich des Flask-Backends diente die KI ebenfalls als Unterstützung beim Aufspüren von Bugs. Sie half dabei, Fehler in der API-Kommunikation, dem Routing und der Datenvalidierung schneller zu identifizieren und zu beheben. Auch hier wurde bei jedem Fehler zunächst ausführlich selbstständig nach Lösungen gesucht.

Frontend-Entwicklung und Refactoring
Im React-Frontend kam Gemini zunächst bei der Fehlersuche im Zusammenhang mit dem CSS-Framework Tailwind zum Einsatz. Da die aufgetretenen Styling- und Konfigurationsprobleme trotz KI-Unterstützung nicht gelöst werden konnten, wurde die Entscheidung getroffen, Tailwind für dieses Projekt vollständig zu verwerfen.

Bei dem darauffolgenden Refactoring übernahm Gemini teilweise die zeitintensive Aufgabe der Code-Übersetzung: Bestehende Tailwind-Klassen in den React-Komponenten wurden durch die KI in React-Inline-Styling umgeschrieben.

Literatur

[1] Alex Teboul. *Diabetes Health Indicators Dataset*. 2021. URL: https://www.kaggle.com/datasets/alexteboul/diabetes-health-indicators-dataset (besucht am 23. 02. 2026).

[2] Cory Dolphin und contributors. *Flask-CORS: A Flask extension for handling Cross Origin Resource Sharing (CORS)*. 2025. URL: https://pypi.org/project/flask-cors/ (besucht am 02. 02. 2026).

[3] Kamijin Fuchi. *React Icons: Include popular icons in your React projects easily*. 2024. URL: https://react-icons.github.io/react-icons/ (besucht am 26. 02. 2026).

[4] Holger Krekel and pytest developers. *pytest: helps you write better programs*. 2025. URL: https://docs.pytest.org/ (besucht am 25. 02. 2026).

[5] Tailwind Labs Inc. *Rapidly build modern websites without ever leaving your HTML*. 2026. URL: https://tailwindcss.com/ (besucht am 26. 02. 2026).

[6] Joblib Development Team. *Joblib v1.5.2*. 2025. URL: https://pypi.org/project/joblib/ (besucht am 24. 02. 2026).

[7] Saurabh Kumar und contributors. *python-dotenv: Read key-value pairs from a .env file and set them as environment variables*. 2025. URL: https://saurabh-kumar.com/python-dotenv/ (besucht am 25. 02. 2026).

[8] Meta Platforms, Inc. *React – The library for web and native user interfaces*. 2025. URL: https://react.dev/ (besucht am 25. 05. 2026).

[9] Michael L. Waskom. *Seaborn v0.13.2*. 2024. URL: https://seaborn.pydata.org/ (besucht am 24. 02. 2026).

[10] Mike Bostock and Observable. *The JavaScript library for bespoke data visualization*. 2026. URL: https://d3js.org/ (besucht am 26. 02. 2026).

[11] Niklas Lang. *F1-Score – einfach erklärt!* 2023. URL: https://databasecamp.de/statistik/f1-score (besucht am 24. 02. 2026).

[12] NumPy Developers. *NumPy v2.3.5*. 2025. URL: https://numpy.org/devdocs/release/2.3.5-notes.html (besucht am 24. 02. 2026).

[13] Pallets. *Flask v3.1.2*. 2025. URL: https://pypi.org/project/Flask/3.1.2/ (besucht am 24. 02. 2026).

[14] Python Developers. *json v2.0.9*. 2025. URL: https://docs.python.org/3/library/json.html (besucht am 24. 02. 2026).

[15] Recharts Group. *Recharts: A composable charting library built on React components*. 2025. URL: https://recharts.org/ (besucht am 26. 02. 2026).

[16] Scikit-learn Developers. *Model persistence: Saving and Loading Machine Learning Models*. 2024. URL: https://scikit-learn.org/stable/model_persistence.html (besucht am 24. 02. 2026).

[17] Scikit-learn Developers. *Scikit-learn 1.8.0*. 2025. URL: https://scikit-learn.org/stable/ (besucht am 24. 02. 2026).

[18] scikit-learn developers. *Random Forest Classifier v1.8.0*. 2025. URL: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html (besucht am 24. 02. 2026).

[19] scikit-learn developers. *Random Forest Classifier v1.8.0*. 2025. URL: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.recall_score.html (besucht am 24. 02. 2026).

[20] Scott Lundberg. *SHAP*. 2017. URL: https://arxiv.org/abs/1705.07874 (besucht am 24. 02. 2026).

[21] The Matplotlib Development Team. *Matplotlib v3.10.8*. 2025. URL: https://matplotlib.org/stable/index.html (besucht am 24. 02. 2026).

[22] The Pandas Development Team. *Pandas v2.3.3*. 2025. URL: https://pandas.pydata.org/docs/dev/whatsnew/v2.3.3.html (besucht am 24. 02. 2026).

[23] Vidhi Chugh. *AUC und die ROC-Kurve beim maschinellen Lernen*. 2024. URL: https://www.datacamp.com/de/tutorial/auc (besucht am 24. 02. 2026).

[24] Matt Zabriskie und contributors. *Axios: Promise based HTTP client for the browser and node.js*. 2025. URL: https://axios-http.com/ (besucht am 26. 02. 2026).